Q. Write a note on Software Evolution. (5 Marks)

📘 Software Evolution

Definition:
Software Evolution is the process of modifying and updating software after its development to meet changing user requirements, fix errors, improve performance, and adapt to new environments.

Software is not developed only once. It needs continuous changes and improvements to remain useful and reliable.

🔹 Reasons for Software Evolution

1. Changing Requirements: User needs and business requirements may change over time.
2. Bug Fixing: Errors and defects in the software must be corrected.
3. Performance Improvement: Software is updated to improve speed, efficiency, and reliability.
4. New Features: New functionalities are added to meet user expectations.
5. Environment Changes: Software must adapt to new operating systems, hardware, and technologies.

🔹 Types of Software Evolution

1. Corrective Maintenance: Fixing errors and bugs.
2. Adaptive Maintenance: Modifying software to work in a new environment.
3. Perfective Maintenance: Improving performance and adding new features.
4. Preventive Maintenance: Making changes to avoid future problems and improve maintainability.

🔹 Importance of Software Evolution

- Increases software reliability.
- Improves user satisfaction.
- Extends the software's life.
- Keeps software up to date with new technologies.

Theerthananda Gowda

💡 Example

A banking application is updated regularly to fix security issues, add UPI payment features, improve performance, and support new banking requirements.

Q. Explain the Challenges in Software Engineering. (6–8 Marks)

📘 1. Introduction

Software Engineering involves developing, maintaining, and managing software systems. During software development, engineers face several challenges that affect the quality, cost, time, and performance of the software.

The major challenges in Software Engineering are explained below.

🔹 2. Challenges in Software Engineering

1. Changing Requirements 🔄

- User requirements may change frequently during software development.
- Developers must modify the software according to new requirements.
- Frequent changes increase development time and cost.

2. Software Complexity ⚙️

- Modern software systems are large and complex.
- Managing different modules, functions, and components is difficult.
- Developers must ensure that all components work together correctly.

3. Time and Cost Management ⏰

- Completing software projects within the given time and budget is challenging.
- Unexpected problems may delay the project.
- Proper planning and resource management are required.

4. Security and Safety 🔒

- Software may face security threats such as hacking, viruses, and data theft.
- Developers must protect sensitive information.

Theerthananda Gowda

- Security testing and proper authentication are important.

5. Quality and Reliability ⭐

- Software must work correctly without errors or failures.
- Maintaining performance, usability, and reliability is challenging.
- Proper testing and debugging are necessary to ensure quality.

6. Maintenance and Upgradation 🔧

- Software requires continuous maintenance after development.
- Developers must fix bugs, improve performance, and add new features.
- Maintaining old software can be difficult and expensive.

7. Rapid Technology Changes 💻

- New programming languages, tools, frameworks, and technologies are introduced frequently.
- Developers must continuously learn and adapt to new technologies.
- Outdated technologies may affect software performance and compatibility.

8. Team Communication and Coordination 🤝

- Large software projects involve multiple developers and teams.
- Poor communication can cause misunderstandings and delays.
- Proper coordination and teamwork are necessary for successful development.

🔹 **3. Conclusion**

Software Engineering faces many challenges, such as changing requirements, complexity, security, cost, quality, and maintenance. Proper planning, communication, testing, and modern development techniques help overcome these challenges and produce high-quality software.

Q. Compare and Contrast Waterfall Model and Prototype Model. (6–8 Marks)

Theerthananda Gowda

📘 1. Introduction

The Waterfall Model and Prototype Model are two software development life cycle (SDLC) models used in software engineering.

- Waterfall Model: A traditional model in which software development follows a step-by-step sequence. Each phase is completed before moving to the next phase.
- Prototype Model: A model in which an initial working version of the software (prototype) is developed to understand and collect user requirements before developing the final system.

🔹 2. Waterfall Model 🌊

- Follows a sequential and linear approach.
- Requirements are collected and finalized at the beginning.
- Each development phase must be completed before starting the next phase.
- Changes are difficult to make once development has started.

Software Development Methodologies: A Deep Dive into the Waterfall Model: Part 2 📊 – The Rails Drop

railsdrop.com

Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance.

🔹 3. Prototype Model 🛠️

- An initial working model of the software is created.
- Users evaluate the prototype and provide feedback.
- Requirements are improved based on user feedback.
- The prototype is refined until requirements are clearly understood, and then the final software is developed.

A Deep Dive into Prototype Model in Software Engineering | Relia Software

reliasoftware.com

Process: Requirements → Quick Design → Build Prototype → User Evaluation → Refinement → Final Product.

🔹 4. Differences Between Waterfall Model and Prototype Model

Basis	Waterfall Model 🌊	Prototype Model 🛠️
1. Approach	Sequential approach.	Iterative approach.
2. Requirements	Requirements are fixed at the beginning.	Requirements are refined using feedback.
3. User Involvement	Less user involvement during development.	High user involvement.
4. Flexibility	Less flexible to changes.	More flexible to changes.
5. Development	Complete system is developed phase by phase.	An initial working model is developed first.
6. Feedback	Feedback is generally received after development or testing.	Feedback is received during prototype development.
7. Cost	Changes at later stages can be expensive.	Early feedback can reduce requirement-related rework.
8. Best Suitable For	Projects with clear and fixed requirements.	Projects with unclear or changing requirements.

🔹 5. Advantages

Waterfall Model:

- Simple and easy to understand.
- Easy to manage due to clearly defined phases.
- Suitable for projects with fixed requirements.

Prototype Model:

- Helps understand user requirements clearly.
- Provides early user feedback.
- Reduces the risk of developing the wrong product.

🔹 6. Conclusion

Both models are useful in software development. The Waterfall Model is suitable when requirements are clear and fixed, whereas the Prototype Model is suitable when requirements are unclear and user feedback is important.

Q. Explain the Characteristics of a Software Process. (5 Marks)

📘 1. Introduction

A Software Process is a set of activities, methods, and steps followed to develop, deliver, and maintain software.

A good software process helps developers produce high-quality software within the given time and budget.

🔹 2. Characteristics of a Software Process

1. Understandability 📖

- The process should be easy to understand.
- Developers should clearly know the activities and steps to be followed.

2. Visibility 👁️

- The progress of software development should be clearly visible.
- Managers should be able to monitor the project's status.

3. Supportability 🛠️

- The process should be supported by tools and techniques.
- Tools help developers perform tasks easily and efficiently.

4. Acceptability 🤝

- The process should be acceptable to the developers and team members.
- It should be practical and easy to follow in real working environments.

5. Reliability 🔒

- The process should help produce software with fewer errors and failures.

Theerthananda Gowda

- It should ensure consistent and dependable results.

6. Robustness 💪

- The process should handle unexpected problems and changes.
- It should continue working effectively even when difficulties occur.

7. Maintainability 🔧

- The process should be easy to modify and improve when requirements change.
- It should support future improvements.

8. Rapidity ⚡

- The process should support faster software development and delivery.
- It should help complete the project within the planned time.

🔹 **3. Conclusion**

A good software process should be understandable, visible, reliable, flexible, and efficient. These characteristics help developers produce quality software and complete projects successfully.

Q. Explain COCOMO Cost Estimation Model. (6 Marks)

📘 **1. Introduction**

COCOMO (Constructive Cost Model) is a software cost estimation model developed by Barry W. Boehm in 1981.

It is used to estimate the effort, development time, and cost required to develop a software project based on its size, measured in KLOC (Thousand Lines of Code).

🔹 **2. Types of COCOMO Models**

COCOMO is divided into three types:

1. Basic COCOMO: Estimates software development effort based only on project size (KLOC).
2. Intermediate COCOMO: Estimates effort based on project size and additional cost factors.

Theerthananda Gowda

3. Detailed COCOMO: Estimates effort for each phase of software development by considering different cost factors.

🔹 3. Project Modes in COCOMO

COCOMO classifies projects into three modes:

1. Organic Mode 🌱: Used for small and simple projects with experienced developers and well-understood requirements. Example: Payroll system.

2. Semi-Detached Mode ⚙️: Used for medium-sized projects with moderate complexity and developers having different levels of experience. Example: Database management system.

3. Embedded Mode 🏢: Used for large and complex projects with strict hardware, software, and operational constraints. Example: Aircraft control system.

🔹 4. Basic COCOMO Formulas

1. Effort Estimation

$$E = a \times (KLOC)^b$$

Where:

- E = Effort in Person-Months.
- KLOC = Thousand Lines of Code.
- a and b = Constants based on project mode.

2. Development Time

$$T = c \times (E)^d$$

Where:

- T = Development time in months.
- c and d = Constants based on project mode.

3. Number of Developers

$$N = \frac{E}{T}$$

🔹 5. COCOMO Coefficient Table

Project Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

🔹 6. Advantages of COCOMO

1. Easy to understand and use.
2. Helps estimate project cost and development time.
3. Supports project planning and resource allocation.
4. Helps project managers manage software projects effectively.

✅ Conclusion

COCOMO is an important software cost estimation model that helps predict the effort, time, and cost required for software development. It helps project managers plan and complete projects effectively.

Q. Explain the Need of an SRS Document. (5 Marks)

📘 1. Introduction

SRS (Software Requirements Specification) is a document that describes all the functional and non-functional requirements of a software system.

It explains what the software should do, how it should behave, and what requirements it must satisfy. It acts as an agreement between the customer and the development team.

🔹 2. Need / Importance of an SRS Document

1. Clear Understanding of Requirements 📖

- Helps developers understand the exact requirements of the customer.
- Reduces confusion and misunderstandings during development.

2. Communication Between Customer and Developer 🤝

- Acts as a common communication document.

Theerthananda Gowda

- Ensures that both customers and developers have the same understanding of the project.

3. Basis for Software Design 📐

- Provides guidelines for designing the software architecture and modules.
- Helps developers plan how the software will be developed.

4. Reduces Development Cost and Time ⏰

- Identifies requirements clearly before development begins.
- Reduces errors, rework, and unnecessary expenses.

5. Supports Software Testing 🧪

- Provides a basis for preparing test cases.
- Helps testers verify whether the software meets the specified requirements.

6. Helps in Software Maintenance 🔧

- Acts as a reference for future modifications and updates.
- Helps developers understand the original system requirements.

🔹 **3. Example**

For an Online Shopping Application, the SRS document specifies requirements such as:

- User registration and login.
- Product searching and selection.
- Online payment.
- Order tracking.

These requirements guide developers during software development.

✅ **4. Conclusion**

An SRS document is an important part of software development because it clearly defines requirements, improves communication, reduces development risks, and ensures that the final software meets customer expectations.

Theerthananda Gowda

Q. Write a Note on Risk Management. (6 Marks)

📘 1. Introduction

Risk Management is the process of identifying, analyzing, monitoring, and controlling risks that may affect a software development project.

A risk is an uncertain event that may cause problems such as project delays, increased cost, software failure, or poor quality.

The main aim of risk management is to reduce the chances of risks and minimize their impact on the project.

🔹 2. Steps Involved in Risk Management

1. Risk Identification 🔍

- Identifying possible risks that may affect the project.
- Example: Lack of skilled developers, changing requirements, and hardware failure.

2. Risk Analysis 📊

- Analyzing the probability and impact of each identified risk.
- Helps determine which risks are more serious.

3. Risk Prioritization ⚖️

- Arranging risks according to their importance and severity.
- High-priority risks are handled first.

4. Risk Planning 📝

- Preparing strategies to avoid or reduce risks.
- Developing backup plans to handle unexpected situations.

5. Risk Monitoring and Control 👀

- Continuously observing risks throughout the project.
- Taking corrective actions whenever a risk occurs.

🔹 3. Types of Risks in Software Engineering

1. Project Risks: Affect project schedule, budget, and resources. Example: Project delay.
2. Technical Risks: Affect software quality and technical performance. Example: Software failure or technology issues.
3. Business Risks: Affect the business success of the software. Example: Loss of funding or market demand.

🔹 **4. Importance of Risk Management**

- Reduces project failure.
- Helps complete projects within budget and time.
- Improves software quality.
- Supports better decision-making.
- Increases project success.

🔹 **5. Example**

Consider an online shopping application. If the payment gateway fails, customers cannot complete payments.

Through risk management, the development team can identify this risk early and prepare a backup payment gateway to avoid service interruptions.

✅ **6. Conclusion**

Risk Management is an essential activity in software engineering. It helps identify potential problems early, reduce their impact, and ensure successful software development within the planned time and budget.

Q. Explain Project Monitoring Plans. (5 Marks)

📘 **1. Introduction**

A Project Monitoring Plan is a document or systematic process used to track and control the progress of a software development project.

It helps project managers check whether the project is progressing according to the planned schedule, budget, resources, and quality standards.

Theerthananda Gowda

The main aim is to identify problems early and take corrective actions to complete the project successfully.

🔹 2. Important Activities of Project Monitoring

1. Schedule Monitoring ⏰

- Tracks whether project activities are completed within the planned time.
- Identifies delays in project tasks.

2. Cost Monitoring 💰

- Monitors the money spent on the project.
- Ensures that the project stays within the estimated budget.

3. Resource Monitoring 👨‍💻

- Checks the proper utilization of developers, tools, hardware, and other resources.
- Ensures that resources are available when needed.

4. Quality Monitoring ⭐

- Checks whether the software meets the required quality standards.
- Monitors testing results, errors, and defects.

5. Risk Monitoring ⚠️

- Identifies new risks and tracks existing risks.
- Helps take preventive and corrective actions.

6. Progress Reporting 📊

- Prepares regular reports about project progress.
- Communicates project status to managers, developers, and customers.

🔹 3. Importance of Project Monitoring Plans

1. Helps complete the project on time.
2. Controls project cost and resources.
3. Identifies problems at an early stage.

Theerthananda Gowda

4. Improves software quality.
5. Supports better project management and decision-making.

🔹 **4. Example**

Consider a Student Management System project.

The project manager regularly checks whether login, student registration, attendance, and report generation modules are completed according to the schedule and budget.

If any module is delayed, corrective actions are taken to complete it on time.

✅ **5. Conclusion**

A Project Monitoring Plan is important in software engineering because it tracks project progress, controls cost and time, monitors quality, and helps identify problems early to ensure successful project completion.

Q. Discuss the Principles of Software Engineering. (7 Marks)

📘 **1. Introduction**

Software Engineering Principles are basic guidelines and rules followed by software developers to develop high-quality, reliable, efficient, and maintainable software.

These principles help developers reduce complexity, avoid errors, save time, and improve software quality.

🔹 **2. Principles of Software Engineering**

1. Modularity 🧩

- Software should be divided into smaller and independent modules.
- Each module performs a specific task.
- Makes software easier to develop, test, and maintain.

2. Abstraction 🎯

- Shows only essential information and hides unnecessary implementation details.
- Reduces complexity and makes software easier to understand.

3. Simplicity ✨

- Software design and code should be simple and easy to understand.
- Avoids unnecessary complexity and reduces errors.

4. Reusability ♻️

- Existing code components should be reused whenever possible.
- Saves development time and effort.
- Improves consistency and reduces duplication.

5. Maintainability 🔧

- Software should be designed so that it can be easily modified and updated.
- Helps developers fix bugs and add new features in the future.

6. Flexibility 🔄

- Software should be designed to handle changing user requirements.
- Makes it easier to modify and extend the system.

7. Reliability 🔒

- Software should perform its functions correctly and consistently.
- It should work without frequent failures under specified conditions.

8. Efficiency ⚡

- Software should use system resources such as memory, processing power, and time effectively.
- Provides better performance to users.

9. Quality Assurance ⭐

- Software should be tested regularly to identify and correct errors.
- Ensures that the final product meets the required quality standards.

🔹 **3. Importance of Software Engineering Principles**

- Improves software quality and reliability.

- Reduces development cost and time.
- Makes software easier to understand and maintain.
- Reduces errors and complexity.
- Improves productivity and customer satisfaction.

✅ 4. Conclusion

Software Engineering Principles provide guidelines for developing high-quality software. Following these principles helps developers create software that is reliable, efficient, reusable, flexible, and easy to maintain.

Q. Describe the Essential Characteristics of a Well-Engineered Software Product. (6 Marks)

📘 1. Introduction

A Well-Engineered Software Product is software that is developed using proper software engineering principles and techniques to meet user requirements.

It should be reliable, efficient, secure, easy to use, and easy to maintain. A good software product provides quality performance and satisfies customer needs.

🔹 2. Essential Characteristics of a Well-Engineered Software Product

1. Maintainability 🔧

- Software should be easy to modify, update, and improve.
- Developers should be able to fix errors and add new features easily.

2. Reliability and Security 🔒

- Software should perform correctly without frequent failures.
- It should protect user data from unauthorized access and security threats.

3. Efficiency ⚡

- Software should use system resources such as memory, processing power, and time effectively.
- It should provide fast performance and response time.

4. Acceptability 👍

- Software should be easy to understand and use.
- It should meet user requirements and work properly in the user's environment.

5. Usability 🖥️

- Software should have a simple and user-friendly interface.
- Users should be able to learn and operate it easily.

6. Flexibility 🔁

- Software should be easy to modify according to changing requirements.
- New features and improvements should be added without major difficulties.

7. Portability 🌐

- Software should be able to run on different operating systems and hardware environments with minimum changes.

8. Scalability 📈

- Software should handle an increase in users, data, and workload without major performance problems.

🔹 **3. Importance**

- Improves customer satisfaction.
- Reduces maintenance costs.
- Increases software reliability.
- Provides better performance.
- Extends the life of the software product.

✅ **4. Conclusion**

A well-engineered software product should have important characteristics such as maintainability, reliability, efficiency, acceptability, and security. These characteristics help produce high-quality software that meets user needs and performs effectively.

Theerthananda Gowda

Q. What are the Needs of Software Process Models? Explain Agile Model in Detail. (8 Marks)

📘 1. Introduction

A Software Process Model is a structured approach that defines the steps and activities involved in developing software, such as requirement analysis, design, coding, testing, and maintenance.

It provides a systematic way to develop high-quality software within the planned time and budget.

🔹 2. Need for Software Process Models

Software process models are needed for the following reasons:

1. Proper Planning 🗓️

- Helps plan and organize software development activities.
- Defines the tasks to be completed at each stage.

2. Better Quality ⭐

- Ensures that software is developed according to quality standards.
- Helps identify and correct errors through proper testing.

3. Time and Cost Management ⏰

- Helps complete projects within the estimated time and budget.
- Reduces unnecessary work and development costs.

4. Risk Management ⚠️

- Helps identify potential risks during software development.
- Allows developers to take preventive actions.

5. Clear Communication 🤝

- Defines the roles and responsibilities of team members.
- Improves coordination between developers, managers, and customers.

6. Easy Maintenance 🔧

- Provides proper documentation and a systematic development structure.
- Makes future modifications and updates easier.

🔹 **3. Agile Model**

Definition:

The Agile Model is an iterative and incremental software development model in which software is developed in small parts called iterations or sprints.

It focuses on continuous customer feedback, teamwork, frequent software delivery, and adapting to changing requirements.

Instead of developing the entire software at once, Agile divides the project into smaller modules and delivers working software regularly.

🔹 **4. Working Process of Agile Model**

Agile QA Process: What It Is, Best Practices - EffectiveSoft

effectivesoft.com

The Agile Model follows these steps:

1. Requirement Gathering 📋

- Collect requirements from customers.
- Identify the features to be developed.

2. Planning 🗓️

- Divide the project into small tasks.
- Decide the features to be completed in each sprint.

3. Design 🎨

- Prepare the design for the selected features.
- Plan the structure and functionality of the software.

4. Development 💻

- Developers write code and develop the selected features.
- Software is built in small increments.

5. Testing 🧪

- Test the developed features to identify errors.
- Fix bugs and ensure software quality.

6. Review and Feedback 🔁

- Demonstrate the working software to customers.
- Collect feedback and make necessary improvements.

7. Deployment 🚀

- Release the completed increment to users.
- Repeat the cycle for the remaining features until the complete product is developed.

🔹 **5. Advantages of Agile Model**

1. Customer Satisfaction: Continuous customer involvement helps meet user expectations.
2. Flexibility: Easily accepts changing requirements during development.
3. Early Delivery: Working software is delivered frequently.
4. Better Quality: Continuous testing helps identify and fix errors early.
5. Reduced Risk: Regular feedback helps identify problems at an early stage.

🔹 **6. Disadvantages of Agile Model**

1. Requires continuous customer involvement.
2. Frequent changes may affect project planning.
3. Less importance may be given to detailed documentation.
4. Difficult to estimate the total cost and completion time for projects with changing requirements.

🔹 **7. Example**

Consider an Online Shopping Application:

- First sprint: User registration and login.
- Second sprint: Product searching and selection.

- Third sprint: Shopping cart and payment.
- Fourth sprint: Order tracking and delivery details.

After each sprint, the customer reviews the working features and provides feedback for improvements.

Q. What is a Process? Explain in Detail the Activities of the Processes. (7 Marks)

📘 1. Definition of Process

A Process is a set of related activities, actions, and tasks performed to achieve a specific goal.

In Software Engineering, a Software Process is a collection of activities followed to develop, deliver, and maintain a software product.

It provides a systematic approach to software development and helps produce high-quality software.

🔹 2. Activities of Software Process

There are four fundamental activities in a software process:

1. Software Specification (Requirement Analysis) 📋

- Defines what the software should do and the requirements of the system.
- Developers collect and analyze customer requirements.
- Both functional and non-functional requirements are identified.
- The requirements are documented in the SRS document.

Example: For a banking application, requirements include user login, money transfer, and balance checking.

2. Software Design and Implementation 🎨💻

- Software design defines the structure and architecture of the system.
- Developers decide how different modules will work together.

Theerthananda Gowda

- Implementation involves writing program code using suitable programming languages.
- The designed system is converted into a working software product.

Example: Designing the login page and writing code for user authentication.

3. Software Validation (Testing) 🧪

- Ensures that the developed software meets customer requirements.
- Software is tested to identify errors and defects.
- Different types of testing, such as unit testing, integration testing, and system testing, are performed.
- Bugs are corrected to improve software quality.

Example: Testing whether users can successfully log in using valid credentials.

4. Software Evolution (Maintenance) 🔧

- Software is modified and updated after delivery.
- New features are added according to changing user requirements.
- Errors are corrected and performance is improved.
- Software is adapted to new technologies and environments.

Example: Adding UPI payment facilities to an existing banking application.

🔹 **3. Diagram of Software Process Activities**

Software Engineering: Dasar, Tujuan, dan Prosesnya 2025 | RevoU

revou.co

🔹 **4. Importance of Software Process Activities**

1. Provides a systematic approach to software development.
2. Improves software quality and reliability.
3. Helps reduce errors and development costs.
4. Ensures customer requirements are satisfied.
5. Makes software maintenance easier.

✅ 5. Conclusion

Software Process consists of four major activities: Software Specification, Software Design and Implementation, Software Validation, and Software Evolution. These activities help develop, test, deliver, and maintain high-quality software systematically.

Q. Write the SRS for the Restaurant Example. (8 Marks)

📘 1. Introduction

SRS (Software Requirements Specification) is a document that describes the functional and non-functional requirements of a software system.

For a restaurant management system, the SRS document explains how the system manages food orders, customers, menus, billing, and payments.

🔹 2. Purpose

- To develop a computerized Restaurant Management System.
- To manage restaurant activities efficiently.
- To reduce manual work and save time.
- To improve customer service and order management.

🔹 3. Scope of the System

The Restaurant Management System provides the following facilities:

- Managing food items and menus.
- Taking customer orders.
- Managing table reservations.
- Generating bills.
- Processing customer payments.
- Maintaining customer and order records.

Theerthananda Gowda

🔹 4. Functional Requirements

Functional requirements describe what the system should do.

1. User Login: The system should allow authorized users to log in.
2. Menu Management: The administrator should be able to add, update, and delete food items.
3. Order Management: Waiters should be able to take and manage customer orders.
4. Table Reservation: Customers should be able to reserve tables.
5. Billing: The system should automatically calculate the total bill.
6. Payment: The system should support cash and online payments.
7. Order Tracking: Staff should be able to check the status of food orders.
8. Report Generation: The system should generate daily sales and order reports.

🔹 5. Non-Functional Requirements

Non-functional requirements describe how the system should perform.

1. Performance: The system should respond quickly to user requests.
2. Security: Only authorized users should access the system.
3. Reliability: The system should work correctly without frequent failures.
4. Usability: The system should have a simple and user-friendly interface.
5. Maintainability: The system should be easy to update and maintain.

🔹 6. Hardware and Software Requirements

Hardware Requirements:

- Computer or laptop.
- Minimum 4 GB RAM.
- Hard disk or SSD.
- Internet connection.

Software Requirements:

- Operating System: Windows or Linux.

- Frontend: HTML, CSS, JavaScript.
- Backend: Java / Python / PHP.
- Database: MySQL.

🔹 **7. Users of the System**

1. Administrator: Manages menus, staff, and reports.
2. Waiter: Takes customer orders and manages tables.
3. Chef: Receives and prepares food orders.
4. Customer: Views menus, places orders, and makes payments.

Q. Explain the Different Software Applications. (7 Marks)

📘 **1. Introduction**

Software Applications are computer programs developed to perform specific tasks and solve real-world problems.

Software applications are used in different fields such as business, education, banking, healthcare, communication, and entertainment.

🔹 **2. Types of Software Applications**

1. Standalone Applications 💻

- These applications run on a computer without requiring a network connection.
- They perform specific tasks independently.
- Example: Calculator, Notepad, and MS Paint.

2. Interactive Transaction-Based Applications 🌐

- These applications allow users to interact with the system and perform transactions.
- They process user requests and provide immediate responses.
- Example: Online shopping, ATM banking, and railway reservation systems.

3. Embedded Control Systems ⚙️

- These applications are built into hardware devices to control their operations.

Theerthananda Gowda

- They perform specific functions automatically.
- Example: Washing machines, microwave ovens, and car control systems.

4. Batch Processing Systems 📊
   - These applications process large amounts of data in groups without requiring continuous user interaction.
   - They are commonly used for repetitive and scheduled tasks.
   - Example: Salary processing, electricity bill generation, and payroll systems.

5. Entertainment Systems 🎮
   - These applications are developed to provide entertainment and recreational activities.
   - They include games, music, videos, and multimedia applications.
   - Example: Video games, music players, and streaming applications.

6. Modeling and Simulation Systems 🔬
   - These applications are used to represent and simulate real-world systems or situations.
   - They help study, analyze, and predict system behavior.
   - Example: Weather forecasting, flight simulators, and scientific simulations.

7. Data Collection Systems 📡
   - These applications collect data from sensors and other devices and store it for analysis.
   - They are used in monitoring and controlling different environments.
   - Example: Weather monitoring systems and traffic monitoring systems.

8. Systems of Systems 🏢
   - These applications combine multiple independent software systems to work together.
   - They provide complex services by sharing information between systems.
   - Example: Air traffic control systems and large banking networks.

🔹 **3. Importance of Software Applications**

- Reduce manual work and save time.
- Improve accuracy and efficiency.
- Support communication and information sharing.
- Automate different business activities.
- Help solve complex real-world problems.

Q. Discuss Agile Model in Detail. (7 Marks)

📘 **1. Introduction**

The Agile Model is an iterative and incremental software development model in which software is developed in small parts called iterations or sprints.

It focuses on continuous customer feedback, teamwork, flexibility, and frequent delivery of working software.

Agile allows developers to make changes easily according to customer requirements during the development process.

🔹 **2. Characteristics of Agile Model**

1. Customer Involvement 🤝 : Customers actively participate and provide regular feedback.
2. Flexibility 🔁 : Easily accepts changing requirements.
3. Incremental Development 📈 : Software is developed and delivered in small parts.
4. Continuous Testing 🧪 : Testing is performed throughout development.
5. Teamwork 👷 : Developers, testers, and customers work together.
6. Frequent Delivery 🚀 : Working software is delivered regularly.

🔹 **3. Working Process of Agile Model**

Agile Software Development Methodology: An In-Depth Guide

cleveroad.com

The Agile Model follows these steps:

1. Requirement Gathering 📋

- Collect requirements from customers.
- Identify the features needed in the software.

2. Planning 🗓️

- Divide the project into smaller tasks.
- Decide which features will be developed in each sprint.

3. Design 🎨

- Prepare the design and architecture for the selected features.
- Decide how the software components will work.

4. Development 💻

- Developers write code and develop the selected features.
- Working software is created in small increments.

5. Testing 🧪

- Test the developed features.
- Identify and correct errors to ensure quality.

6. Review and Feedback 🔄

- Demonstrate the developed software to customers.
- Collect feedback and make necessary changes.

7. Deployment 🚀

- Release the completed software increment to users.
- Repeat the cycle until all required features are developed.

🔹 **4. Advantages of Agile Model**

1. Provides flexibility to changing requirements.

2. Improves customer satisfaction through continuous feedback.
3. Delivers working software frequently.
4. Helps identify errors at an early stage.
5. Reduces development risks.

🔹 **5. Disadvantages of Agile Model**

1. Requires continuous customer involvement.
2. Frequent changes may affect project planning.
3. Documentation may receive less attention.
4. Difficult to estimate the exact project cost and completion time.

🔹 **6. Example**

Consider an Online Shopping Application:

- Sprint 1: User registration and login.
- Sprint 2: Product searching and selection.
- Sprint 3: Shopping cart and online payment.
- Sprint 4: Order tracking and delivery details.

After each sprint, the customer reviews the working features and provides feedback for improvements.

Q. What are the Reasons to Have a Model? (5 Marks)

📘 **1. Introduction**

A Software Model is a simplified representation of a software system or process. It describes the structure, design, and working of a system before actual development.

Models help developers understand, plan, design, and develop software more effectively.

🔹 **2. Reasons to Have a Model**

1. Easy Understanding 📖

- Models make complex software systems easier to understand.

- They provide a clear representation of the system.

2. Better Communication 🤝

- Models help developers, customers, and project managers communicate effectively.
- Everyone gets a common understanding of the system.

3. Reduces Complexity ⚙️

- Models divide complex systems into smaller and simpler parts.
- This makes software design and development easier.

4. Early Error Detection 🔍

- Models help identify design mistakes and missing requirements before coding.
- This reduces development time and cost.

5. Better Planning and Design 📐

- Models provide a blueprint for software development.
- They help developers plan system structure and functionality.

6. Documentation 📝

- Models provide a visual and written reference for future development.
- They help during software maintenance and modifications.

🔹 **3. Example**

Before developing an online shopping application, developers create UML diagrams to represent customer login, product selection, payment, and order processing.

This helps them understand the system before writing code.

Q. Explain the Components of SRS. (7 Marks)

📘 **1. Introduction**

SRS (Software Requirements Specification) is a document that describes the complete requirements of a software system.

Theerthananda Gowda

It specifies what the software should do, how it should perform, and the conditions it must satisfy. It acts as an agreement between the customer and the development team.

🔹 2. Components of SRS

An SRS document mainly consists of the following components:

1. Introduction 📖

- Provides an overview of the software system.
- Describes the purpose, scope, and objectives of the project.
- Identifies the intended users of the document.

2. Overall Description 🌐

- Describes the general structure and working of the software.
- Explains product perspective, product functions, and user characteristics.
- Identifies assumptions and dependencies.

3. Functional Requirements ⚙️

- Describe the functions and services that the software must provide.
- Explain how the system should respond to user inputs.
- Example: User login, registration, searching, and payment.

4. Non-Functional Requirements ⭐

- Describe the quality attributes and performance of the software.
- Include security, reliability, usability, efficiency, and maintainability.
- Example: The system should respond within a few seconds.

5. External Interface Requirements 🔗

- Describe how the software interacts with external systems and users.
- Include user interfaces, hardware interfaces, software interfaces, and communication interfaces.
- Example: Connecting an application to a payment gateway.

6. System Constraints ⚠️

- Define the limitations and restrictions of the software system.
- Include hardware, software, programming language, and security constraints.
- Example: The application must support Windows operating systems.

7. Assumptions and Dependencies 📝

- Describe conditions assumed to be true during software development.
- Identify external factors on which the system depends.
- Example: The application requires an active internet connection.

🔹 **3. Example**

For an Online Shopping Application, the SRS document specifies:

- Functional requirements: Login, product search, shopping cart, and payment.
- Non-functional requirements: Security, performance, and reliability.
- Interface requirements: User interface and payment gateway connection.

Q. What is Scheduling? Explain the Different Types of Scheduling in Detail. (7 Marks)

📘 **1. Introduction**

Scheduling is the process of planning and organizing project activities by assigning start dates, end dates, time durations, and resources to each task.

In Software Engineering, scheduling helps project managers complete software development activities within the planned time and budget.

The main objective of scheduling is to ensure that all project activities are completed in the correct order and on time.

🔹 **2. Different Types of Scheduling Techniques**

1. Gantt Chart Scheduling 📊

- A Gantt Chart is a graphical representation of project activities against time.
- It displays the start date, end date, and duration of each task.
- It helps project managers track project progress.

Theerthananda Gowda

- It is easy to understand and use.

Example: Planning the requirements, design, coding, and testing phases of a software project.

What Is a Gantt Chart & How to Make One? (with Examples)

deltek.com

2. PERT (Program Evaluation and Review Technique) 🗓️

- PERT is a scheduling technique used to estimate the time required to complete project activities.
- It represents activities in a network diagram.
- It considers three time estimates:
    - Optimistic Time (O): Minimum expected time.
    - Most Likely Time (M): Normal expected time.
    - Pessimistic Time (P): Maximum expected time.

Expected time formula:

$$T_E = \frac{O + 4M + P}{6}$$

- It is useful for projects where activity durations are uncertain.

3. CPM (Critical Path Method) ⏳

- CPM is a scheduling technique used to identify the longest sequence of dependent activities in a project.
- This sequence is called the Critical Path.
- It determines the minimum time required to complete the project.
- Any delay in critical path activities delays the entire project.

Example: Requirement Analysis → Design → Coding → Testing → Deployment.

4. Milestone Scheduling 🎯

- Milestone scheduling identifies important events or checkpoints in a project.
- It helps measure project progress at important stages.

- Milestones do not represent task durations; they represent significant achievements.

Example:

- Completion of requirement analysis.
- Completion of software design.
- Completion of coding.
- Final project submission.

🔹 **3. Importance of Scheduling**

1. Helps complete projects within the planned time.
2. Improves resource utilization.
3. Helps identify project delays.
4. Improves coordination among team members.
5. Supports effective project management.

🔹 **4. Example**

Consider a Student Management System project:

Activity	**Duration**
Requirement Analysis	1 Week
System Design	1 Week
Coding	3 Weeks
Testing	1 Week
Deployment	1 Week

Scheduling helps the team organize these activities and complete the project within 7 weeks.